

Raport Techniczny: Miniprojekt Bazodanowy

Enterprise Desk Management System (EDMS)

Maja Drohobycka, Krzysztof Rodzinka

11 czerwca 2026 r.

Spis treści

1	Wstęp i cel projektu	3
2	Wykorzystane technologie	3
3	Opis modelu danych (Schemat)	3
3.1	Struktura tabel	3
4	Realizacja operacji w bazie danych	4
4.1	Przetwarzanie transakcyjne (ACID)	4
4.2	Raportowanie i widoki	4
4.3	Automatyzacja - Triggery	4
5	Dyskusja i wydajność	4
6	Instrukcja uruchomienia	5
7	Dane dostępne do testów	5

1 Wstęp i cel projektu

Celem projektu było zaprojektowanie i zaimplementowanie fragmentu aplikacji bazodanowej służącej do zarządzania rezerwacjami stanowisk pracy (biurek) w nowoczesnej przestrzeni biurowej. System został zaprojektowany pod kątem wymagań korporacyjnych, uwzględniając podział na budynki, piętra, departamenty oraz role użytkowników (Administrator i Pracownik).

2 Wykorzystane technologie

Do realizacji projektu wykorzystano następujący stos technologiczny:

- **Silnik Bazy Danych:** SQLite – relacyjna baza danych wybrana ze względu na pełną wsparcie dla transakcji ACID oraz łatwość przenoszenia projektu (baza w jednym pliku .db).
- **Język programowania:** Python 3.11+ – serce logiki biznesowej i backendu.
- **Interfejs (Frontend):** Streamlit – biblioteka umożliwiająca stworzenie dynamicznego interfejsu webowego.
- **Zasilanie danymi:** Faker – generator syntetycznych danych testowych.
- **Analiza danych:** Pandas – do obsługi ramek danych i prezentacji wyników zapytań SQL.

3 Opis modelu danych (Schemat)

Model danych składa się z 14 tabel powiązanych relacjami, co zapewnia pełną normalizację danych:

3.1 Struktura tabel

1. **Słowniki (S_):** S_Typy_Biurek, S_Statusy_Rezerwacji, S_Statusy_Usterki – zapewniają spójność danych.
2. **Struktura organizacyjna:** Budynki, Pietra, Departamenty, Pracownicy – hierarchia firmy i uwierzytelnianie.
3. **Zasoby:** Biurka (z opisem strefy, np. "Strefa Ciszy") oraz Wyposazenie.
4. **Operacje:** Rezerwacje (kluczowa tabela faktów), Usterki oraz Zamiany (tabela asocjacyjna dla systemu wymiany biurek).
5. **Audyt:** Logi_Systemowe.

4 Realizacja operacji w bazie danych

4.1 Przetwarzanie transakcyjne (ACID)

Najważniejszą operacją w systemie jest proces rezerwacji, który został zrealizowany wewnątrz bloku transakcyjnego. System przed zatwierdzeniem (*COMMIT*) sprawdza następujące warunki:

- Czy wybrane biurko jest sprawne (*czy_sprawne* = 1).
- Czy termin (dzień i godzina) nie nakłada się na inną rezerwację tego samego biurka.

Logika walidacji nakładania się przedziałów czasowych:

```
1 SELECT COUNT(*) FROM Rezerwacje
2 WHERE id_biurka = ? AND data_rezerwacji = ?
3 AND (godzina_start < ? AND godzina_koniec > ?)
```

4.2 Raportowanie i widoki

Zaimplementowano zaawansowane zapytania raportujące (Views), które automatyzują analitykę biurową:

- **VW_RAPORT_OBLOZENIA:** Widok łączący 4 tabele, obliczający procent zajętych stanowisk w każdym budynku na bieżący dzień.
- **VW_USTERKOWOSC:** Raport grupujący usterki według biur, ułatwiający planowanie serwisu.

4.3 Automatyzacja - Triggery

Zastosowano wyzwalacze (*Triggers*) do automatyzacji procesów:

- **TR_AUTO_AWARIA:** Reaguje na dodanie wpisu w tabeli *Usterki*, automatycznie ustawiając flagę *czy_sprawne* = 0 w tabeli *Biurka*.
- **TR_LOG_REZERWACJA:** Rejestruje historię operacji w tabeli audytowej.

5 Dyskusja i wydajność

W celu weryfikacji wydajności systemu, baza została zasilona ponad **11 000 rekordów** przy pomocy generatora *Faker*.

- **Współbieżność:** Dzięki zastosowaniu klauzuli *BEGIN TRANSACTION* i blokowaniu rekordów, system poprawnie obsługuje jednoczesne prośby o rezerwację bez ryzyka nadpisania danych (tzw. *double booking*).
- **Optymalizacja:** Zapytania raportujące wykorzystują agregację (*COUNT*, *ROUND*) i złączenia, wykonując się w czasie poniżej 100ms.
- **Kaskadowość:** Zastosowanie *ON DELETE CASCADE* w tabeli *Zamiany* gwarantuje spójność bazy przy usuwaniu rezerwacji przez administratora.

6 Instrukcja uruchomienia

1. Sklonować repozytorium: `git clone https://github.com/krzysiuro/Bazy-Danych.git`
2. Zainstalować wymagane biblioteki: `pip install -r requirements.txt`
3. (Opcjonalnie) Wygenerować dane testowe: `python generator.py`
4. Uruchomić aplikację: `streamlit run app.py`

7 Dane dostępowe do testów

- **Administrator:** admin@firma.pl | hasło: admin123
- **Pracownik:** adam@firma.pl | hasło: adam123